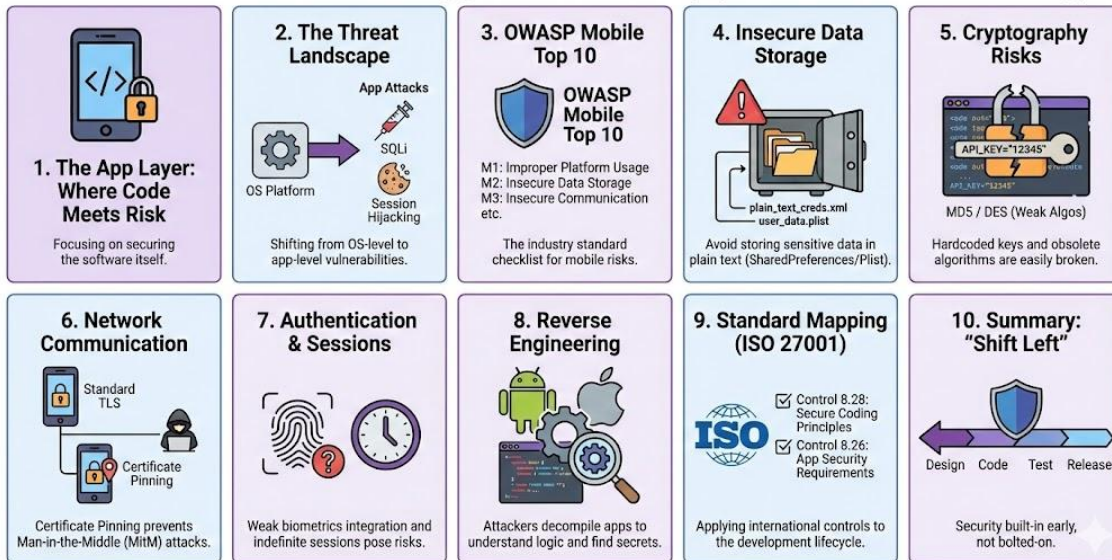


CMT 451 – Mobile Operating Systems Security

Module 3: Mobile Application Security

CMT 451 Module 3: Mobile Application Security (10-Slide Visual Summary)



Module Context and Learning Objectives

Module 3 shifts the focus of mobile security from the operating system and hardware layer to the application layer. While Android and iOS have significantly improved sandboxing, secure boot, and hardware-backed security, most real-world mobile breaches continue to occur at the application level due to insecure coding practices and flawed design assumptions.

By the end of this module, students should be able to identify common mobile application attack vectors, evaluate risks using the OWASP Mobile Top 10, recommend secure coding practices for Android and iOS, and explain how application security maps to ISO/IEC 27001 controls.

Mobile Application Threat Landscape

Mobile applications operate in hostile environments where attackers may have full control of the device. Threat actors include physical attackers with access to lost or stolen devices, network attackers operating rogue Wi-Fi access points, reverse engineers who decompile application binaries, and logical attackers who exploit broken authentication or session management.

Unlike server environments, mobile apps must assume the client is untrusted. Any secret stored, computation performed, or decision made on the client can potentially be observed, modified, or replayed by an attacker.

OWASP Mobile Top 10 – Risk-Based Perspective

The OWASP Mobile Top 10 provides a risk-based prioritization of the most critical mobile application security risks. It is not a checklist but a reflection of real-world attack prevalence, impact, and exploitability.

Key risks include improper platform usage such as misuse of biometric APIs, insecure data storage where secrets are stored in plaintext, and insecure communication due to weak TLS implementation or lack of certificate pinning.

Secure Storage and Secret Management

Insecure data storage remains the most common mobile application failure. Developers often assume that mobile devices are inherently secure and store sensitive information such as passwords, tokens, or PINs in SharedPreferences, Info.plist files, or local databases without proper protection.

Secure storage requires both encryption and key protection. Android Keystore and iOS Keychain provide hardware-backed key storage, access control, and resistance to offline extraction attacks.

Authentication, Sessions, and Network Security

Biometric authentication mechanisms such as fingerprint or facial recognition should be treated as user interface controls rather than authentication logic. Backend systems must independently validate cryptographic tokens and enforce session expiration and revocation.

Insecure communication exposes applications to Man-in-the-Middle attacks. Certificate pinning reduces the trust surface by ensuring the application only trusts a specific server certificate or public key.

Reverse Engineering, Tampering, and Shift-Left Security

Mobile applications are publicly distributed artifacts that can be decompiled and analyzed. Defensive techniques such as code obfuscation, runtime integrity checks, and root or jailbreak detection raise the cost of attack but do not provide absolute protection.

Shift-Left Security emphasizes integrating security requirements, threat modeling, and secure coding practices early in the software development lifecycle. This approach aligns with ISO/IEC 27001 requirements for secure engineering.

Case Study: The Secure Phone, the Insecure App

This case study examines a mobile banking breach where attackers exploited insecure token storage, misuse of biometrics, lack of certificate pinning, and poor session management. The operating system remained uncompromised.

Students are encouraged to analyze which design decisions failed, how earlier security reviews could have prevented the breach, and how governance controls could enforce secure development practices.

More Details

presentation Goal: To demonstrate that a secure OS cannot compensate for insecure application code.

+1

I. Technical Deep Dive (75 Minutes)

- **The Threat Pivot:** While OS attacks are becoming harder due to sandboxing, app attacks are increasing due to developer mistakes and time pressure.

+1

- **OWASP Mobile Top 10 Focus:**
 - **M2: Insecure Data Storage:** Developers often mistakenly store credentials in SharedPreferences (Android) or Info.plist (iOS) in plain text.

+2

- **The Correct Approach:** Use the **Android Keystore System** or **iOS Keychain Services**, which provide hardware-backed encryption.

+1

- **Network & Reverse Engineering:**

- **Certificate Pinning:** Moving beyond standard TLS to trust only specific server certificates to block MitM attacks.

+4

- **Obfuscation:** Using code-mangling techniques to slow down attackers who decompile APK/IPA files.

+1

- **Governance Mapping:** Aligning app security with **ISO/IEC 27001** controls 8.26 (Requirements) and 8.28 (Secure Coding).

+2

II. Case Study: The "Leaky Wallet" Incident (45 Minutes)

- **Scenario:** A fintech startup releases a mobile wallet. To speed up performance, developers store OAuth tokens in a local SQLite database in plain text. They rely on standard TLS for transfers.

+1

- **The Attack:** An attacker performs a **Man-in-the-Middle (MitM)** attack at a public cafe by installing a malicious Root CA on a user's phone.

+1

- **The Failure:** Because the app did not implement **Certificate Pinning**, it trusted the attacker's proxy. The attacker then extracted the local database from the device, gaining full account access.

+2

- **Resolution Exercise:** Students must map these failures to the OWASP Top 10 (M2, M3) and recommend the **Shift-Left** principle for the next patch.